- **Black-box / White-box** → based on **knowledge of the internal code**
- **Smoke / Sanity / Regression / Retesting** → based on **purpose and scope**
- **Exploratory** → based on **how testing is performed**

Then after a new build arrives:

```
NEW BUILD
   ↓
SMOKE
   ↓
Is the build stable enough to test?
   ↓
SANITY
   ↓
Did the specific change/fix work?
   ↓
REGRESSION
   ↓
Did the change break existing functionality?
```

And if a specific bug was fixed:

```
BUG
 ↓
DEVELOPER FIXES IT
 ↓
RETESTING
 ↓
Does the exact bug now work?
```

Now let's go one by one.

1️⃣ Black-Box Testing

Simple definition

> **Testing the software without knowing or depending on its internal source code.**

You focus on:

```
INPUT → SOFTWARE → OUTPUT
```

You don't care how the developer implemented it.

What can you test?

- Functional behavior
- UI
- API behavior
- Input validation
- Error messages
- Business rules
- Integration behavior

Your previous techniques fit very nicely here:

```
Equivalence Partitioning
Boundary Value Analysis
Decision Tables
State Transition Testing
```

These are commonly used as **black-box test design techniques**.

2️⃣ White-Box Testing

Now flip the situation.

You **do know the internal code/logic.**

You test the internal structure of the software.

🧠 What does White-Box focus on?

Things like:

- Code paths
- Branches
- Conditions
- Loops
- Statements
- Internal logic

Common white-box techniques include:

Statement Coverage

Did every statement execute?

Branch Coverage

Did every decision branch execute?

Interview answer

Black-box testing validates software behavior without requiring knowledge of its internal implementation**, whereas white-box testing** examines internal code structure and logic.

As a QA Automation Engineer, you will primarily encounter **black-box/system/API/UI testing**, but understanding white-box concepts is still valuable. "White-box testing is not only done by developers."

QA engineers can perform code-aware testing, coverage analysis, unit/integration-level testing, etc., depending on the team and role.

3️⃣ Smoke Testing 🔥

"Is this build stable enough for serious testing?"

Example

New build:

```
Veeva Application v5.2
```

You perform a small set of critical tests:

```
Can application launch?
Can user login?
Can user logout?
Can user create document?
Can user save document?
Can user search document?
```

If everything is catastrophically broken:

```
Login ❌
Application crashes ❌
Database unavailable ❌
```

You don't waste 6 hours executing 500 detailed test cases.

You reject the build.

4️⃣ Sanity Testing :

A developer fixes:

> **Document upload is failing for PDF files.**

A new build arrives.

You don't necessarily need to run the entire application.

First, focus on the affected functionality.

```
Upload PDF
Upload invalid PDF
Upload large PDF
Upload corrupted PDF
```

You're asking:

> **"Does the recent change/fix appear to work correctly?"**

That's sanity testing.

5️⃣ Regression Testing :

Regression testing verifies that recent changes have not adversely affected existing functionality.

You modify:

```
Payment module
```

Regression might include:

```
Login
Cart
Checkout
Payment
Refund
Order history
Notifications
```

You don't necessarily test every feature equally.

You prioritize based on:

- Impact
- Risk
- Dependencies
- Changed components
- Critical business flows

Imagine:

2,000 regression test cases Running manually every sprint is painful and expensive. So you automate stable, repeatable scenarios:

Selenium , REST Assured , TestNG/JUnitCl/ , CD

6️⃣ **Retesting :**

BUG-123: PDF upload fails for files larger than 50 MB.

Developer says:

"Fixed."

You execute the same failed scenario again.